

# PsiMeter: An Adversarially-Auditable Protocol for Large-Scale, Anonymous Online Tests of Putative Psi Effects

Adler Oliveira  
The PsiMeter Project  
Independent Research  
psimeter.org  
adleroliveira@psimeter.org

**Abstract**—Research on putative “psi” effects, such as intention-correlated bias of a true random number generator (micro-psychokinesis) and forced-choice anticipation of future random outcomes (precognition), has produced decades of small and contested results. We think their weak standing owes less to their nominal statistics than to a familiar set of methodological liberties (optional stopping, selective reporting, flexible baselines, undisclosed multiple comparisons) and, beneath those, to an unavoidable reliance on the experimenter’s good faith. PsiMeter is an open protocol and reference platform that sets out to remove that reliance. It treats the server and whoever runs it as untrusted, so that any third party can reconstruct every published result from public data alone. Before a session begins, the operator’s declared intention, the experiment parameters, and a pseudonymous key are sealed into a hash commitment. The session is then bound to a signed public-randomness beacon, which fixes the moment it could first have existed, and the raw output of a physical entropy source is committed through a streaming Merkle tree while the generator reads nothing back from the client. Everything is written to an append-only, externally time-stamped ledger, and independent witnesses may co-sign each step as it happens. Public per-operator evidence is reported as an e-value, a test martingale that stays valid even though players stop whenever they please. We give the threat model, the construction, the statistical design, and the verification procedure an auditor would follow, and we are explicit about the trust that remains. The instrument is built to be as capable of a clean null as of a positive result. No confirmatory data has been collected; this paper is an invitation to scrutinize the method, re-implement the verifier, and collaborate.

**Index Terms**—anomalous cognition, micro-psychokinesis, precognition, true random number generator, pre-registration, commitment scheme, randomness beacon, Merkle tree, transparency log, anytime-valid inference, e-values, test martingale, verifiable computation, reproducibility

## I. INTRODUCTION

People have been testing the idea that human intention or anticipation can nudge a physical random process for the better part of a century [1], [2], and the question is still open. The trouble is not that the reported effects are large and refuse to replicate. It is that they are **small**, and small effects are exactly where method matters most. A per-bit micro-psychokinesis effect on the order of  $10^{-4}$  [3], [4] is invisible in any single run and surfaces only after a great deal of pooling. That is the regime in which optional stopping, selective reporting, a poorly chosen baseline, or a few undisclosed comparisons can conjure a signal out of nothing or wash a real one away. The

same liberties are now blamed for irreproducibility across the empirical sciences [5], [6].

A second problem cuts deeper, and it is peculiar to a contested claim: the reader has to trust the experimenter. A flawless pre-registered analysis is still only as good as one’s confidence that the data were gathered as described, that no flattering runs were quietly dropped, that timestamps were not adjusted after the fact, and that the code which ran is the code that was published. In established fields, institutional reputation and routine replication carry that weight. For a claim as extraordinary, and as reputationally costly, as psi, ordinary trust is neither available nor enough.

Our position is that this trust problem can be engineered away rather than argued away. We treat the experimenter, the server, and the host as untrusted, and we require that every claim of consequence be reconstructible by recomputation from public artifacts. An auditor holding nothing but the published ledger should be able to confirm five things: that each session’s intention and parameters were fixed before any relevant randomness existed; that the raw data were neither altered nor cherry-picked; that the session demonstrably came after a known public moment; that nothing in the corpus was inserted, removed, reordered, or back-dated; and that every published number can be reproduced from the raw data by an open, deterministic procedure. None of these checks may depend on believing the operator.

The contribution of this paper is to show that such an instrument can be both adversarially auditable and, at the same time, a gamified, public, anonymous, planet-scale data collector. We state a threat model in which the experimenter is untrusted, and we split a reviewer’s trust into a cryptographic **integrity path** and a statistical **inference path** (Section III). We then specify the protocol that realizes it: the canonical encoding, the commitments, beacon-anchored freshness, a hash-chained ledger, one-way isolated generation with a streaming output commitment, and independent live witnesses (Sections V through XII). Two experiment kinds, binary micro-psychokinesis and forced-choice presentiment, share one provenance spine (Section X). For the public score we adopt an anytime-valid test martingale, which is what makes a continuously updated, optional-stopping-prone leaderboard statistically defensible (Section XI). Finally, a conformance regime built on

machine-checkable test vectors lets anyone re-implement the protocol and re-derive the results (Section XIII).

A word on what this manuscript is. It describes version 0.1.0-draft of the PsiMeter protocol as of June 2026, and it reports a method, not a finding. No confirmatory data has been collected, and we advance no claim that psi exists. The instrument is meant to be as good at producing a convincing null as a convincing positive, and throughout we treat a well-powered null as a primary scientific result. The reference implementation, the specification, and the test vectors are open source.

## II. RELATED WORK

*Anomalies research.* The modern micro-psychokinesis (micro-PK) paradigm comes from the Princeton Engineering Anomalies Research (PEAR) program, which ran large numbers of fixed-length trials on a hardware random event generator while an operator held an intention toward higher or lower output [2], [7]. Meta-analyses of the wider literature find a small aggregate effect alongside clear heterogeneity and signs of small-study bias [3], [4], and the Global Consciousness Project carried the same instrumentation into the field [8]. Forced-choice anticipation is older still [1], and it returned to prominence, and controversy, with Bem’s “feeling the future” experiments [9] and the meta-analysis that followed [10]; physiological “presentiment” ahead of randomly chosen affective images has its own meta-analytic record [11]. The two experiment kinds in Section X are deliberately conservative descendants of these paradigms.

*The methodological critique.* The sharpest analyses of this field are not appeals to prior implausibility but inventories of researcher degrees of freedom: mis-specified baselines, optional stopping, and selective reporting [4], [12], and in the case of [9] the kind of flexible analysis that inflates false-positive rates [6]. The same decade brought the broader reckoning with reproducibility in psychology [5] and the rise of pre-registration and Registered Reports [13]. We read that critique as a design brief and answer each item with a mechanism: pre-commitment, fixed- $N$  generation, an empirically calibrated baseline, a hash-bound analysis, and a split between an exploratory and a confirmatory phase.

*Cryptographic provenance.* The integrity machinery is ordinary, well-worn cryptography rather than anything new. Merkle trees [14] are the backbone of modern transparency logs, Certificate Transparency above all [15], whose append-only, gossip-checkable design is the model for our ledger. Freshness comes from a public randomness beacon: drand [16], run by the League of Entropy, emits threshold-BLS [17] signed pulses on a fixed schedule, building on earlier bias-resistant randomness work [18]. Long-term timestamping uses RFC 3161 tokens [19] and Bitcoin-anchored OpenTimestamps [20], [21]. Canonicalization, hashing, and signatures follow JCS [22], SHA-256 [23], and Ed25519 [24].

*Anytime-valid inference.* The public score rests on betting-based statistics. A test martingale has expectation one under the null, so by Ville’s inequality [25] its running maximum exceeds  $1/\alpha$  with probability at most  $\alpha$ . That single fact licenses continuous monitoring and data-dependent stopping, the long-standing hazard of sequential testing [26], without inflating the error rate. The recent theory of e-values and anytime-valid inference [27], [28], [29], [30] turns this into a practical tool, and Good’s weight of evidence [31] supplies the deciban scale we use for display. As far as we know, PsiMeter is the first anomalies instrument whose public, gamified, per-participant score is an e-value, which is precisely what lets a leaderboard coexist with a valid false-positive guarantee (Section XI).

## III. THREAT MODEL AND DESIGN PRINCIPLES

### A. The untrusted-experimenter axiom

A credible result must not ask anyone to trust the server operator, the host, or the operator’s code. This one requirement shapes everything that follows. Each load-bearing claim is made checkable by recomputation from published artifacts, so that the operator’s honesty never enters a reviewer’s conclusion.

### B. The two trust paths

It helps to separate a reviewer’s trust into two paths. The first is the integrity path, which is cryptographic: capture raw bits, commit to them, sign, hash-chain, and log. It is checked by recomputation (Section XIII), and it does not depend on the language anything was written in, because SHA-256, Ed25519, and the Merkle construction behave the same everywhere. An auditor re-runs them and trusts no particular implementation. The second is the inference path, which is statistical: every confirmatory number is produced by an open, deterministic analysis over the published raw data, never by the live server, and the figures shown on screen are for display only. Because the server lives entirely in the first path and never in the second, the choice of server implementation carries no scientific risk. Fig. 1 sketches the resulting layout.

### C. One-way isolation

While a micro-PK run is generating, the random process reads nothing from the client. The isolation is structural rather than a matter of policy, so a client, even a malicious or scripted one, cannot bias an individual run; at most it can add to the volume of sessions, which the analysis treats separately. Precognition has to be interactive, and it stays sound for a different reason: each target is tied to a future beacon round that no one can predict at the moment of choice (Section X-B).

### D. Design pillars

Seven pillars guide the construction. The experimenter is untrusted. Predictions are committed before the data exist, not chosen afterward. Provenance is complete and tamper-

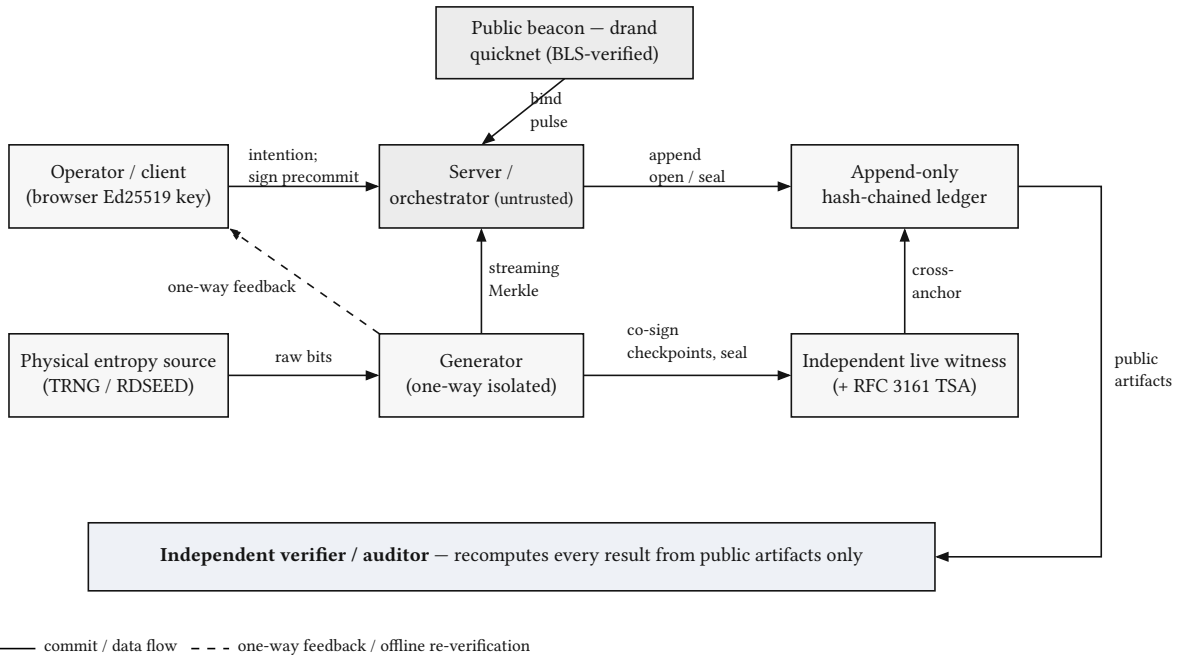


Fig. 1: System architecture and the two trust paths. The server is untrusted. The micro-PK generator is one-way isolated: no edge returns to it from the client, so the channel cannot be used to bias a run. Solid edges are commit and data flow that an auditor recomputes; the dashed edge is one-way visual feedback. Every published result can be reconstructed by an independent verifier from public artifacts.

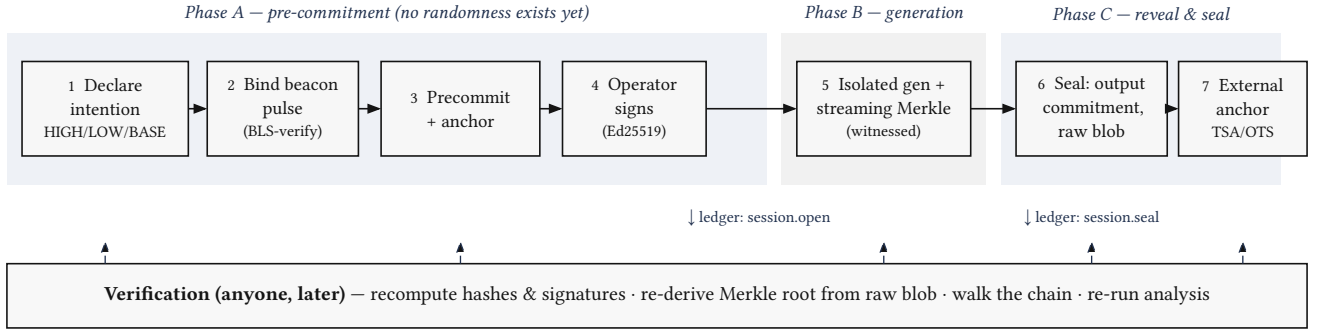


Fig. 2: The provenance spine of a micro-PK session. In Phase A every operator decision is frozen before any relevant randomness exists. Phase B generates a fixed number of raw samples under one-way isolation while committing them through a streaming Merkle tree. Phase C seals the output commitment and externally anchors the ledger head. The same spine carries every experiment kind; only the choice vocabulary, the generation and reveal protocol, and the scoring change. Any party can later re-derive every value from the published artifacts.

evident. Analysis is reproducible from raw data. The generator is isolated. Inference is honest at the level of the whole corpus, so that engagement features can never be mistaken for evidence. And the code, protocol, pre-registration, and eventual data are open. These translate into six concrete goals, namely independent verifiability, independent implementability, pre-commitment, freshness, tamper-evidence, and cross-language byte-parity, which the rest of the paper makes good on.

#### IV. PROTOCOL OVERVIEW

PsiMeter is a handful of components with sharply separated roles: an operator’s client holding a browser key, an untrusted server, an isolated generator, a physical entropy source, a public beacon, an append-only ledger, one or more independent witnesses, and any number of verifiers. They follow a single provenance spine, shown in Fig. 2: commit, sign, bind a beacon,

generate and commit, seal, hash-chain, anchor. Sections V through XII specify each stage.

The normative specification from which this paper draws assigns every requirement a permanent identifier such as PSI-MERKLE-3, cited by the reference code and tests so that the implementation stays bound to the document; we refer to those areas here without restating every clause. All hashing and signing run over one canonical byte encoding (Section V). That single discipline is what makes byte-identical commitments from independent implementations possible, and machine-checkable.

## V. CANONICALIZATION AND CRYPTOGRAPHIC PRIMITIVES

### A. Canonical encoding

Everything that gets hashed or signed is first put into PsiMeter Canonical JSON (PCJ), a restricted profile of the JSON Canonicalization Scheme [22]. Object members are sorted by their UTF-16 code units, the separators are the bare `,` and `:`, no insignificant whitespace survives, and strings use minimal escaping. The one rule that does real work is on numbers: every number must be a finite integer in the safe range  $-(2^{53} - 1)$  to  $2^{53} - 1$ , and non-integers are rejected outright. This is on purpose. Every committed quantity in PsiMeter is an integer or a string, and the real-valued statistics (z-scores, e-values) are all derived later and never appear in a committed payload. Banning floating point from the committed surface removes the only genuine cross-language hazard in JSON canonicalization, its number formatting, and it is what lets a TypeScript generator and a Python verifier produce the same hashes to the byte. Absent members are omitted rather than written as null, so a payload can gain new optional fields later without disturbing the hash of older values that lack them.

### B. Hashing, tagging, and signatures

The hash is SHA-256 [23]. Digests travel as tagged strings of the form `sha256:<64-hex>`, and keys and signatures as `ed25519:<hex>`, so a stored value always says what it is and can be swapped for a different algorithm later without ambiguity. A structured value is hashed as  $H(\text{PCJ}(v))$ , the hash of its canonical bytes. Signatures are Ed25519 [24], and by convention they are taken over the UTF-8 octets of the referenced message string, usually a `sha256:` hash string, signed as text rather than over raw digest bytes, so each context names the exact string it signs. Table I lists the primitives and their encodings.

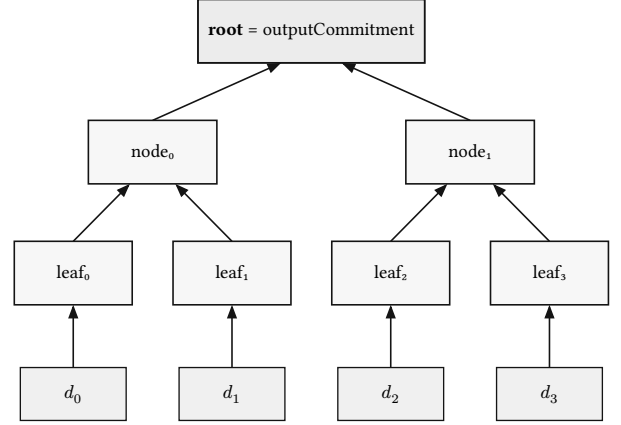
TABLE I: CRYPTOGRAPHIC PRIMITIVES AND THEIR DOMAIN-SEPARATED ENCODINGS.

| Construction   | Definition                                                                  | Encoding                      |
|----------------|-----------------------------------------------------------------------------|-------------------------------|
| Canonical form | RFC 8785 subset; sorted keys, compact separators, integers and strings only | UTF-8 octets                  |
| Hash $H(x)$    | SHA-256 of octet string $x$                                                 | <code>sha256:+64 hex</code>   |
| Merkle leaf    | $H(0x00 \parallel d)$                                                       | <code>sha256:...</code>       |
| Merkle node    | $H(0x01 \parallel l \parallel r)$                                           | <code>sha256:...</code>       |
| Public key     | Ed25519 verifying key                                                       | <code>ed25519:+64 hex</code>  |
| Signature      | Ed25519 over UTF-8 of the message hash string                               | <code>ed25519:+128 hex</code> |

### C. Merkle commitment with domain separation

An ordered list of leaves is committed with a binary Merkle tree [14] that separates its two domains explicitly. A leaf over octet string  $d$  is  $H(0x00 \parallel d)$ , and an internal node over child

digests  $l$  and  $r$  is  $H(0x01 \parallel l \parallel r)$  (Fig. 3). The distinct prefixes make it impossible to pass a leaf preimage off as an internal node. When a level has an odd count, the last node is promoted unchanged; the Bitcoin habit of duplicating it is forbidden, since that admits a duplicate-leaf ambiguity. The same tree serves both the streaming output commitment of a run and the progressive checkpoint roots emitted during generation (Section IX).



$$\text{leaf}(d) = H(0x00 \parallel d) \quad \text{node}(l, r) = H(0x01 \parallel l \parallel r)$$

an odd final node is promoted unchanged – no Bitcoin-style last-node duplication

Fig. 3: The streaming output commitment is a domain-separated Merkle tree over fixed-size windows of the raw entropy stream. Re-deriving the root from the published raw blob and comparing it against the sealed value catches any alteration or reordering of the data.

## VI. EXPERIMENT DEFINITIONS AND PRE-COMMITMENT

### A. Content-addressed experiment definitions

A session runs under an experiment definition: a versioned, immutable, content-addressed parameter set made of an identifier, an integer version, a kind, a block of integer and string parameters, and the allowed choice vocabulary. Its hash,  $\text{experimentHash} = H(\text{PCJ}(\text{definition}))$ , pins the exact parameters without copying them into the commitment. A definition cannot change once published. Any edit, however small, has to bump the version, which changes the hash, and two sessions whose definitions hash differently are never pooled in one confirmatory analysis. Configurability therefore cannot turn into a hidden degree of freedom: every parameter change is visible, time-stamped in the ledger, and splits the corpus.

### B. The pre-commitment and the human anchor

Before any randomness relevant to a session exists, the operator's decision and the full context of the session are frozen into one hash, the pre-commitment. It is taken over a canonical object that binds the experiment id, version, and hash, the committed intention, the operator's public key, the bound beacon pulse (Section VII), a session id, a server nonce, and the current ledger head:  $\text{precommit} =$

$H(\text{PCJ}(\text{PrecommitInput}))$ . Because PCJ sorts members, this hashes a canonical object rather than a brittle concatenation of fields. The first 60 bits of the digest become a short human-readable anchor of twelve Crockford base-32 symbols [32], shown as xxxx-xxxx-xxxx. The anchor is a fingerprint a person can check by eye, and it fingerprints the whole commitment, the intention and parameters and identity and freshness together, not some “seed”. The operator records it, often as a screenshot, as their own proof of what was committed. In the gamified interface the anchor doubles as the large central focus the operator concentrates on, so the scientific artifact and the ritual the player performs are one and the same.

### C. Operator authentication

The operator signs the pre-commitment with a browser-held Ed25519 key, signing the UTF-8 octets of the precommit hash string. The signature gives non-repudiation in both directions: the server cannot edit the intention afterward without breaking it, and the operator cannot later disown what they signed. This pseudonymous key holds no personal data, yet it lets an operator build up a history of sessions, which is the unit of analysis for the individual-consistency hypothesis (Section XI). Keys belonging to the experimenter role are declared in advance and excluded from the confirmatory subject pool, the strongest guard available against the experimenter-as-subject problem.

## VII. FRESHNESS: PUBLIC-BEACON BINDING

Every confirmatory session binds a pulse from a public randomness beacon, fetched at commitment time, into the committed input. Since the pulse did not exist before it was published, the session provably could not have been computed earlier. That closes off a server which pre-generates a library of runs and keeps only the flattering ones.

The confirmatory beacon is drand “quicknet” [16], an unchained threshold-BLS scheme on a three-second period operated by the League of Entropy [18]. Before binding a pulse, an implementation has to verify its BLS signature [17] against the hardcoded group public key and must refuse a pulse that fails. The endpoint serving the pulse is never trusted for authenticity; only the signature is. Because the scheme is unchained, the signed message is simply SHA-256(round) over the eight-octet big-endian round number, with no dependence on any previous signature, so verification stands on its own. The chain hash and group key are published constants an auditor can check against the drand parameters. A development beacon may be used offline, but it is non-confirmatory: it gives no freshness guarantee, and sessions bound to it never enter a confirmatory corpus.

## VIII. TAMPER-EVIDENCE: THE LEDGER

### A. Hash-chained envelope

The ledger is an append-only, hash-chained log of every entry. Each entry carries a sequence number, an informational timestamp, the previous entry’s hash, a type, and a type-specific payload, and is named by  $\text{entryHash} = H(\text{PCJ}(\{\text{seq}, \text{ts}, \text{prevHash}, \text{type}, \text{payload}\}))$ . The first entry has sequence zero and a fixed genesis predecessor; each later entry has a sequence one greater and a prevHash equal to its predecessor’s entryHash. To check the chain, an auditor walks it in order and verifies, for every entry, the expected sequence, the prevHash linkage, and the recomputed entryHash. A single failure breaks the chain there and the corpus is rejected. This one mechanism catches insertion, deletion, reordering, and any tampering with a payload. The wall-clock timestamp is informational; the time that is actually trusted is the bound beacon round, not the server’s clock.

TABLE II: LEDGER ENTRY TYPES. A SIBLING FEED CARRIES `witness.attest` ENTRIES (SECTION XII).

| Entry type      | What it commits                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------|
| genesis         | chain origin                                                                                               |
| session.open    | pre-commitment, operator key and signature, bound beacon, entropy-source metadata, anchor                  |
| session.seal    | output commitment (Merkle root), raw-blob reference, flat SHA-256, integer counts; optional witness fields |
| baseline.seal   | operator-absent calibration run (excluded from scoring)                                                    |
| external.anchor | ledger head with RFC 3161, git, and OpenTimestamps proofs                                                  |
| witness.anchor  | cross-reference to the independent witness-feed head                                                       |

### B. Entry types and payloads

A session is a `session.open` entry followed, once generation finishes, by a `session.seal` entry (Table II). The open entry publishes the pre-commitment, the operator key and signature, the bound beacon, a verbatim descriptor of the entropy source, the server nonce, and the anchor. The seal entry publishes the output commitment (the Merkle root over the raw stream), a content-addressed reference to the persisted raw blob, the flat SHA-256 of the whole stream, the Merkle leaf size, and the integer counts (for instance, the number of one-bits out of the total) from which display statistics are later derived. A seal payload may not carry a derived real-valued statistic such as a z-score; only integers are stored, and the statistics are recomputed at analysis time. The optional witness fields (Section XII) are simply absent on an un-witnessed seal, which is then byte-for-byte its old self.

### C. External anchoring

From time to time an `external.anchor` entry publishes the current ledger head together with independent timestamp proofs, namely an RFC 3161 token [19], a public Git commit, and

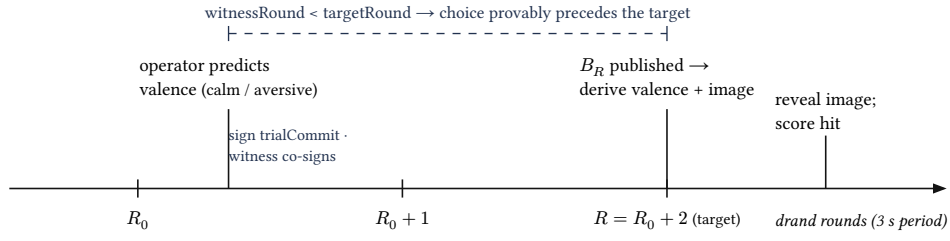


Fig. 4: Presentiment timing. The operator predicts the upcoming stimulus’s valence and signs a per-trial commitment before the target beacon round  $R = R_0 + \text{offset}$  is published; an independent witness co-signs while  $R$  is still future ( $\text{witnessRound} < \text{targetRound}$ ). When  $B_R$  appears it derives the valence (an exact, beacon-derived fair coin) and the image index. Because the target cannot be known at the moment of choice, the necessarily interactive channel is sound.

a Bitcoin-anchored OpenTimestamps proof [20]. The whole corpus is thereby frozen in time against an adversary who controls the server. A `witness.anchor` entry binds the head of the independent witness feed into the main chain.

## IX. GENERATION AND OUTPUT COMMITMENT

Once the open entry is logged, the generator produces the raw stream and commits to it. Four properties are mandatory. The generation is one-way isolated: during a micro-PK run the generator reads no input from the client. The sample size is fixed and there is no optional stopping: exactly the number of trials set by the hash-bound parameters is generated, with no early stop and no extension, and the analysis uses the complete run. The samples are raw and unconditioned, taken as the source delivers them and never whitened, since conditioning could scrub out the very signal under test; the exact source is recorded in the open entry. And the commitment is streaming: the stream is cut into consecutive leaf windows, one Merkle leaf each, and the root over the prefix so far is emitted as a checkpoint root, the final root being the output commitment. The full stream is persisted and content-addressed, and a verifier later recomputes both the flat hash and the Merkle root from the blob and checks them against the seal. The mapping from raw octets to trials is fixed by the definition and never changes after publication.

Statistical validity depends only on collecting the fixed number of independent samples, not on any regularity in wall-clock time. The stream can therefore be paced for the human experience, giving the visualization its cadence, without the pacing interval being a committed parameter. Event-loop jitter or a garbage-collection pause can disturb the animation, never the bit counts.

## X. EXPERIMENT KINDS

Every kind shares the provenance spine of Sections V through X and the witness protocol of Section XII, and they differ only in the choice vocabulary, the generation and reveal protocol, and the scoring. Two are defined.

### A. Binary micro-psychokinesis

The operator declares HIGH, LOW, or BASELINE before the run, and a fixed-length stream of raw bits is then generated under one-way isolation. The confirmatory contrast is HIGH against LOW, which cancels any static bias of the source, while BASELINE runs are operator-absent calibration and are left out of scoring. A trial is a fixed block of raw bits, and the recorded statistic is just the integer count of one-bits. The published version-1 parameters (Table III) follow the PEAR benchmark protocol [2]. They are values of a published definition, not constants of the protocol, so any change bumps the version and the hash.

TABLE III: PUBLISHED binary-micro-pk v1 PARAMETERS, FOLLOWING THE PEAR BENCHMARK.

| Parameter                                               | Value                 |
|---------------------------------------------------------|-----------------------|
| trialBits (one trial $\sim \text{Binomial}(200, 1/2)$ ) | 200                   |
| bitRatePerSec                                           | 1000                  |
| sessionSeconds                                          | 180                   |
| trialsPerSession                                        | 900                   |
| bitsPerSession                                          | 180 000               |
| checkpointEveryTrials                                   | 5                     |
| intentionAssignment                                     | volitional            |
| conditioning                                            | none                  |
| intentions                                              | HIGH / LOW / BASELINE |

### B. Forced-choice presentiment

Here the operator predicts, before the trial’s target exists, the valence of an affective image they are about to see: calm or aversive. The target is derived entirely from a future beacon round, so neither side can know it at the moment of choice, and that is what keeps the necessarily two-way channel sound (Fig. 4). Given the future round’s randomness  $B_R$ , the valence is a single bit,  $\text{SHA-256}(\text{bytes}(B_R) \parallel \text{uint32}_{\text{be}}(\text{idc}))[\text{0}] \wedge 1$ , which is an exact fair coin whatever the pool sizes are, and a disjoint slice of the same digest, taken modulo the pool size, picks the specific image from a content-hash-pinned corpus. At choice time, with  $R_0$  the latest published round, the operator binds and signs a commitment to the future target round  $R = R_0 + \text{offset}$ ; the fact that  $R > R_0$  shows the target was still future when the choice was made. After the round



publishes, its pulse is BLS-verified, the valence and image are derived, and a hit is scored when the prediction matches. Each trial is re-verifiable offline and in the browser: an auditor re-derives the valence and image index from  $B_R$ , confirms the shown image against the committed manifest, re-hashes the served image bytes against the committed digest, checks each signature and the  $R > R_0$  ordering, and recomputes the per-session Merkle root. The chain runs from beacon to committed image hash to real pixels to valence to hit. Deriving the target from the beacon alone, rather than from any server-held entropy, is a choice that buys reproducibility: anyone can reproduce a trial, with no trust in the server at all.

## XI. STATISTICAL METHODOLOGY

### A. Two hypotheses, two phases

The platform tests two pre-registered hypotheses, kept apart because they call for different analyses. H1, individual consistency, does not ask whether anomalies occur, since they will by chance, but whether particular operators deviate above chance again and again across their own sessions; its clean unit is the within-operator HIGH minus LOW difference, backed by split-half test-retest reliability. H2, excess corpus deviation, asks whether the distribution of session scores departs from an empirically calibrated null in mean, variance, or tail weight, always measured against operator-absent baseline runs through the identical pipeline rather than an assumed  $N(0, 1)$ . Both sit inside a two-phase design. An open exploratory firehose generates hypotheses and flags candidates but is never cited as confirmation. A confirmatory corpus then fixes the hypotheses, the sample size, and the stopping rule, with an analysis script whose hash is anchored before the data exist. For H2 we pre-register a leave-one-operator-out and a per-operator-capped analysis, so that no single heavy or dishonest user can manufacture the corpus result.

### B. Display statistics are not authoritative

The numbers on screen are for display only. The micro-PK per-session display statistic is  $z = (k - n/2)/\sqrt{n/4}$  for  $k$  one-bits out of  $n$ , the precognition statistic is the matching hit-rate  $z$ , and independent sessions combine by Stouffer's method [33],  $Z = (\sum z_i)/\sqrt{k}$ . None of these ever enters a committed payload, and the authoritative figures are recomputed from the raw data by the open analysis pipeline (Section XIII).

### C. The psi score: an anytime-valid e-value

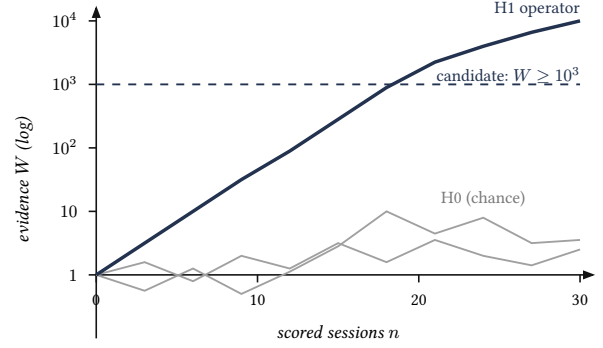


Fig. 5: Illustrative test-martingale trajectories. Under H0 the wealth  $W$  is a non-negative martingale with  $\mathbb{E}[W] = 1$ , so by Ville's inequality it rarely reaches the candidate threshold  $W \geq 10^3$ . An H1 operator with a consistent effect in the declared direction accrues wealth steadily. Ranking even by peak wealth keeps the false-positive guarantee.

The public per-operator score has to update live after every session, and players will naturally walk away on a good run. That is operator-level optional stopping, the very thing fixed- $N$  testing forbids. The way out is to make the score a test martingale, an e-value, which is anytime-valid: under the null its wealth  $W$  is a non-negative martingale with  $\mathbb{E}[W] = 1$ , so by Ville's inequality [25]

$$\mathbb{P}\left(\sup_t W_t \geq 1/\alpha\right) \leq \alpha. \quad (1)$$

One can therefore watch it continuously, stop at will, and even rank a leaderboard by peak wealth, and the false-positive guarantee still holds [27], [29].

Each scored session contributes a directional z-score  $d_i$ : micro-PK HIGH gives  $+z$  and LOW gives  $-z$ , BASELINE is excluded, and precognition is already oriented. Under H0,  $d_i \sim N(0, 1)$ . With  $S = \sum_{i=1}^n d_i$ , the wealth is the equal-weight, one-sided mixture over a frozen grid  $\delta \in \{0.1, 0.2, 0.4, 0.8\}$ ,

$$W = \frac{1}{J} \sum_{j=1}^J \exp(\delta_j S - n\delta_j^2/2), \quad J = 4, \quad (2)$$

computed in log-space for stability. Each component is a martingale because  $\mathbb{E}[\exp(\delta Z - \delta^2/2)] = 1$  for  $Z \sim N(0, 1)$ , so the mixture is one too, and any fixed grid is valid; the grid changes the power, not the validity. The score is shown in readable units: decibans of evidence [31], points =  $\max(0, \lfloor 10 \log_{10} W \rfloor)$ , an anytime-valid p-value  $\min(1, 1/W)$ , and a sigma-equivalent  $\Phi^{-1}(1 - 1/W)$ .

Candidate status is a screening flag, never a proof. An operator is flagged candidate only on  $W \geq 10^3$  together with at least five scored sessions, which asks for consistency rather than one lucky run. The flag means “selected for a separate, pre-registered, fixed- $N$  confirmatory replication”, and the leaderboard has to show the expected-by-chance candidate count beside it. With many players some candidates are expected, which is the whole reason a candidate has to replicate. This

is our structural answer to the multiple-comparisons critique [4]: the open firehose is a hypothesis-generating screen, and its flags are redeemed only in the confirmatory phase. Fig. 5 shows the behavior.

#### D. An honest note on power

The canonical micro-PK effect is roughly  $1$  to  $2 \times 10^{-4}$  per bit. A single 180 000-bit session has a proportion standard deviation near  $1.2 \times 10^{-3}$ , so an effect of canonical size adds only about  $z \approx 0.13$  to one session, which is negligible. All of the real inference therefore lives in aggregation across many sessions and operators. We state this plainly rather than bury it, because it is the reason the platform is built for scale, the reason a single high-sigma session is never evidence, and the reason H1 asks each candidate to contribute many sessions.

### XII. LIVE WITNESSES

Two attacks survive everything above, because the server alone produces every artifact. The first is parallel runs in micro-PK: a malicious server could privately roll several physical streams for one pre-committed session and seal only the flattering one. The second is choice-timing, or back-dating, in precognition: a server could lie about when a forced choice arrived, so that it no longer provably precedes its target. Independent live witnesses, co-signing artifacts as they happen, close both.

A witness is an independent process with its own Ed25519 key. It co-signs a subject and publishes the co-signature on its own append-only, hash-chained feed. The signed statement binds the subject hash, the session and, for per-trial subjects, the trial index, the witnessed kind, and a self-verified drand round, so the time anchor cannot be detached from what it attests. For micro-PK the witness co-signs the open, every checkpoint root, and the seal, and the sealed output commitment must be the Merkle continuation of the witnessed checkpoint prefixes, so no privately rolled alternate stream can be substituted at the seal. For precognition the witness co-signs each forced-choice commitment while its target round is still future, refusing otherwise, which makes the choice provably precede the target. The trusted witness set and a threshold  $M$  are an input the auditor supplies from their own published list, not something the server asserts, and they may be pinned into the hash-bound definition for confirmatory use. The protocol is  $M$ -of- $N$  capable; the reference deployment runs at  $N = 1$ . Trusted time is layered: a self-verified drand round on each attestation, an RFC 3161 token [19] on the feed head, and Bitcoin-anchored OpenTimestamps [20] for the long term.

### XIII. VERIFICATION PROCEDURE AND CONFORMANCE

The protocol says, normatively, what a conforming verifier does with a ledger and the artifacts it references. Every check works on public artifacts alone and trusts neither the server nor any one implementation. In order, a verifier recomputes

each hashed or signed value through the canonical encoding and the primitives; verifies chain integrity, meaning the sequence, the prevHash linkage, and each entryHash; re-derives each experimentHash, pre-commitment, and anchor, and checks the operator signature; confirms each bound beacon pulse against the public archive and BLS-verifies it, and confirms that the open precedes the seal; recomputes the flat hash and the Merkle output commitment from the raw blob, and, for witnessed micro-PK, checks that the seal root continues the witnessed prefixes; for precognition, re-derives each valence and image, re-hashes the served pixels, verifies each signature and the round ordering, and recomputes the trial Merkle root; for witnessed sessions, recomputes each witness statement, verifies each signature, enforces the timing rule, and confirms the quorum of distinct trusted keys; confirms the ledger head against its external-anchor proofs; and recomputes every published statistic, the display  $z$  and the per-operator e-value, from the raw data. Any failure rejects the corpus.

There are two reference verifiers: an offline checker in Python that uses only the standard library, and an in-browser verifier that recomputes the pre-commitment, anchor, and signatures on the client side. Conformance is defined by two classes. A verifier performs every required check and reproduces every test vector. An instrument, or generator, emits only artifacts a verifier accepts and reproduces every test vector. Conformance is therefore something a machine can check, not a claim. A suite of known-answer test vectors covers canonicalization, the primitives, the experiment hash, the pre-commitment, the ledger, presentiment, the psi score, and the witness statement, and it is loaded as shared fixtures by both the reference generator's test suite and the independent verifier, so continuous integration fails the moment either drifts from the frozen values. This is how cross-language byte-parity is enforced: independent implementations are required to agree, byte for byte, on the same public artifacts.

### XIV. SECURITY CONSIDERATIONS AND RESIDUAL TRUST

It is better to state the residual trust plainly than to imply there is none.

*Witness independence.* Witnesses help only to the extent they are independent of the experimenter. Against a server that colludes with every witness, only an  $M$ -of- $N$  quorum of genuinely independent witnesses helps. The protocol supports  $M$ -of- $N$ , but the reference deployment runs at  $N = 1$ , and there the un-forgable time root is the RFC 3161 timestamp authority, with Bitcoin behind it for the long term, rather than the node itself, so back-dating is bounded to timestamp granularity even then. Un-witnessed sessions are never pooled with witnessed confirmatory data, and the strength of the guarantee grows with the number of independent peers running witnesses. The reference repository invites anyone to run one.



*Entropy-source integrity.* Whether the hardware is genuinely physical and unmanipulated is a separate question from the cryptographic chain, and it is handled separately, by empirical baseline calibration, by continuous randomness test batteries such as NIST SP 800-22, Dieharder, and TestU01, and by open, auditable hardware, not by hashing. A seeded pseudo-random generator is kept out of the confirmatory micro-PK arm, since a deterministic stream offers nothing to influence.

*Published code versus running code.* Verification assumes the code that was published is the code that ran. Open source and reproducible build hashes mitigate this, but the strongest mitigation is independent implementations: a verifier built from the specification, agreeing with this one on the same public ledger. That is the entire point of the conformance regime.

*Static bias and Sybil resistance.* A one-sided-intention operator could in principle gather e-value from a biased source rather than from psi. Balancing HIGH and LOW, and centering the confirmatory analysis on the calibrated baseline and the HIGH minus LOW contrast, blunts this; precognition is immune, since its target is an exact beacon-derived fair coin. Minting many operator keys buys more screening tickets, which is answered in the confirmatory phase, and by optional stronger identity for flagged candidates, rather than on the open firehose, and by showing the expected-by-chance candidate count.

## XV. IMPLEMENTATION AND AVAILABILITY

The reference stack splits along the trust paths of Section III. The correctness-critical logic, namely canonicalization, hashing, the Merkle tree, ledger chaining, commitments, and scoring, sits in a pure, I/O-free core that is exhaustively unit-tested. A server orchestrates sessions, generation, the ledger, beacon binding, and anchoring. An independent witness node co-signs in real time. A small entropy-provider sidecar exposes raw, unconditioned bytes from a chosen source. A separate analysis pipeline runs the deterministic, authoritative statistics. The entropy ladder runs from operating-system randomness, which is non-confirmatory plumbing, through the CPU's on-die thermal-noise instruction (RDSEED, a genuine physical source good enough for piloting), to open-hardware USB true-RNGs and, in the end, a quantum RNG as the confirmatory target. Every session records the exact source, device, and parameters verbatim.

Engagement is the data-collection engine, since sample size is what powers H1 and H2, so PsiMeter is also a public, gamified website. The gamification never alters the protocol or misrepresents what a result means. The leaderboard ranks people, not sessions, by the anytime-valid score, and there is deliberately no "most extreme single session" list, because such anomalies are guaranteed by chance and prove nothing. The specification text is licensed CC BY 4.0 and the reference code is MIT, so anyone can build an interoperable generator or an independent verifier; the test vectors ship with the specification.

## XVI. DISCUSSION

PsiMeter's central claim is about method, not about parapsychology. A contested empirical question can be moved out of "trust the experimenter" and into "recompute it yourself". The same construction that defends against a dishonest experimenter also defends against self-deception, which for a hypothesis this laden with prior belief is the more likely failure. The design is symmetric in the outcome it can produce. Because no confirmatory statistic is computed by the server, because the analysis is hash-anchored before the data exist, and because the experimenter is excluded from the confirmatory pool, a null will be exactly as credible, and as publishable, as a positive. We regard a well-powered, cleanly verified null as a primary success.

The approach has limits worth saying out loud. The cryptography certifies provenance and integrity, not physics: on its own it cannot prove that a given hardware source is genuinely indeterministic, which is why source integrity is handled separately and conservatively. At a single witness, the independence guarantee leans on an external timestamp authority rather than on a federation. And the open firehose is only a screen; nothing on the leaderboard is evidence until it has replicated under the frozen confirmatory protocol. None of this is hidden, and each is an explicit line in the residual-trust account.

The pattern generalizes beyond psi. Any field where a small, contested effect has to be measured by an interested party, and where the raw measurements can be committed, time-anchored, and re-analyzed, can adopt the same separation of an integrity path from an inference path. We offer the protocol, the reference implementation, and the conformance vectors in that spirit, and we ask for critique, independent reimplementations of the verifier, adversarial review of the threat model, and collaboration on the confirmatory phase.

## XVII. CONCLUSION

We have described PsiMeter, a verifiable-experiment protocol that makes large-scale, anonymous online tests of putative psi effects auditable by anyone, on the explicit assumption that the experimenter is untrusted. By committing every operator decision before the relevant randomness exists, anchoring freshness to a signed public beacon, committing raw physical entropy through a streaming Merkle tree under one-way isolation, recording everything in an externally anchored hash-chained ledger, admitting independent live witnesses, and reporting public evidence as an anytime-valid e-value, the protocol removes the methodological liberties that have made this literature so hard to adjudicate. The instrument is outcome-neutral, its results are reproducible from public artifacts, and its specification, code, and test vectors are open. What remains is the empirical work, and the scrutiny of others, which this paper exists to invite.

## ACKNOWLEDGMENT

This work builds on public-good infrastructure: the drand and League of Entropy beacon, the OpenTimestamps project, and the open standards it composes (JCS, Ed25519, RFC 3161, Certificate Transparency). It also owes a debt to the literatures on methodological reform and on anytime-valid inference, which together made a design that is gamified yet rigorous seem possible at all. The author thanks in advance the scientists and engineers who will scrutinize, re-implement, and improve this protocol.

## REFERENCES

- [1] J. B. Rhine, *Extra-Sensory Perception*. Boston Society for Psychic Research, 1934.
- [2] R. G. Jahn, "The Persistent Paradox of Psychic Phenomena: An Engineering Perspective," *Proceedings of the IEEE*, vol. 70, no. 2, pp. 136–170, 1982, doi: 10.1109/PROC.1982.12260.
- [3] D. I. Radin and R. D. Nelson, "Evidence for Consciousness-Related Anomalies in Random Physical Systems," *Foundations of Physics*, vol. 19, no. 12, pp. 1499–1514, 1989.
- [4] H. Bösch, F. Steinkamp, and E. Boller, "Examining Psychokinesis: The Interaction of Human Intention with Random Number Generators—A Meta-Analysis," *Psychological Bulletin*, vol. 132, no. 4, pp. 497–523, 2006, doi: 10.1037/0033-2909.132.4.497.
- [5] Open Science Collaboration, "Estimating the Reproducibility of Psychological Science," *Science*, vol. 349, no. 6251, p. aac4716, 2015, doi: 10.1126/science.aac4716.
- [6] E.-J. Wagenmakers, R. Wetzels, D. Borsboom, and H. L. J. van der Maas, "Why Psychologists Must Change the Way They Analyze Their Data: The Case of Psi," *Journal of Personality and Social Psychology*, vol. 100, no. 3, pp. 426–432, 2011, doi: 10.1037/a0022790.
- [7] R. G. Jahn and B. J. Dunne, "On the Quantum Mechanics of Consciousness, with Application to Anomalous Phenomena," *Foundations of Physics*, vol. 16, no. 8, pp. 721–772, 1986.
- [8] P. Bancel and R. Nelson, "The GCP Event Experiment: Design, Analytical Methods, Results," *Journal of Scientific Exploration*, vol. 22, no. 3, pp. 309–333, 2008.
- [9] D. J. Bem, "Feeling the Future: Experimental Evidence for Anomalous Retroactive Influences on Cognition and Affect," *Journal of Personality and Social Psychology*, vol. 100, no. 3, pp. 407–425, 2011, doi: 10.1037/a0021524.
- [10] D. Bem, P. Tressoldi, T. Rabeyron, and M. Duggan, "Feeling the Future: A Meta-Analysis of 90 Experiments on the Anomalous Anticipation of Random Future Events," *F1000Research*, vol. 4, p. 1188, 2015, doi: 10.12688/f1000research.7177.2.
- [11] J. Mossbridge, P. Tressoldi, and J. Utts, "Predictive Physiological Anticipation Preceding Seemingly Unpredictable Stimuli: A Meta-Analysis," *Frontiers in Psychology*, vol. 3, p. 390, 2012, doi: 10.3389/fpsyg.2012.00390.
- [12] J. Utts, "Replication and Meta-Analysis in Parapsychology," *Statistical Science*, vol. 6, no. 4, pp. 363–378, 1991.
- [13] C. D. Chambers, "Registered Reports: A New Publishing Initiative at Cortex," *Cortex*, vol. 49, no. 3, pp. 609–610, 2013, doi: 10.1016/j.cortex.2012.12.016.
- [14] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87*, in LNCS, vol. 293. Springer, 1988, pp. 369–378.
- [15] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, 2013.
- [16] League of Entropy, "drand: A Distributed Randomness Beacon Daemon." 2023.
- [17] D. Boneh, B. Lynn, and H. Shacham, "Short Signatures from the Weil Pairing," *Journal of Cryptology*, vol. 17, no. 4, pp. 297–319, 2004, doi: 10.1007/s00145-004-0314-9.
- [18] E. Syta *et al.*, "Scalable Bias-Resistant Distributed Randomness," in *2017 IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 444–460. doi: 10.1109/SP.2017.45.
- [19] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato, "Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)," RFC 3161, 2001.
- [20] P. Todd, "OpenTimestamps: Scalable, Trust-Minimized, Distributed Timestamping with Bitcoin." 2016.
- [21] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System." 2008.
- [22] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, 2020.
- [23] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180–4, 2015.
- [24] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, 2017.
- [25] J. Ville, *Étude critique de la notion de collectif*. Paris: Gauthier-Villars, 1939.
- [26] A. Wald, "Sequential Tests of Statistical Hypotheses," *Annals of Mathematical Statistics*, vol. 16, no. 2, pp. 117–186, 1945.
- [27] A. Ramdas, P. Grünwald, V. Vovk, and G. Shafer, "Game-Theoretic Statistics and Safe Anytime-Valid Inference," *Statistical Science*, vol. 38, no. 4, pp. 576–601, 2023, doi: 10.1214/23-STS894.
- [28] G. Shafer, "Testing by Betting: A Strategy for Statistical and Scientific Communication," *Journal of the Royal Statistical Society: Series A*, vol. 184, no. 2, pp. 407–431, 2021, doi: 10.1111/rssa.12647.
- [29] P. Grünwald, R. de Heide, and W. M. Koolen, "Safe Testing," *Journal of the Royal Statistical Society: Series B*, vol. 86, no. 5, pp. 1091–1128, 2024, doi: 10.1093/jrsssb/qkae011.
- [30] V. Vovk and R. Wang, "E-Values: Calibration, Combination and Applications," *Annals of Statistics*, vol. 49, no. 3, pp. 1736–1754, 2021, doi: 10.1214/20-AOS2020.
- [31] I. J. Good, "Weight of Evidence: A Brief Survey," in *Bayesian Statistics 2*, J. M. Bernardo, M. H. DeGroot, D. V. Lindley, and A. F. M. Smith, Eds., North-Holland, 1985, pp. 249–270.
- [32] D. Crockford, "Base32 Encoding." 2019.
- [33] S. A. Stouffer, E. A. Suchman, L. C. DeVinney, S. A. Star, and R. M. Williams, *The American Soldier: Adjustment During Army Life*, vol. 1. Princeton University Press, 1949.